

CAP 4

1. Conceito de Processo

Um processo é definido como um **programa em execução**, sendo uma entidade ativa que deve progredir de forma sequencial. Enquanto o programa é uma entidade passiva (como um arquivo em disco), o processo inclui componentes dinâmicos :

- **Contador de Programa:** Indica a próxima instrução a ser executada.
- **Pilha (Stack):** Armazena dados temporários como parâmetros de funções e variáveis locais.
- **Seção de Dados:** Contém as variáveis globais do programa.

2. Estados de um Processo

Durante seu ciclo de vida, um processo transita por diferentes estados conforme as instruções são executadas ou eventos ocorrem :

- **Novo (New):** O processo está sendo criado.
- **Em Execução (Running):** Instruções estão sendo processadas pela CPU.
- **Em Espera (Waiting):** O processo aguarda um evento externo, como o término de uma operação de entrada/saída (I/O).
- **Pronto (Ready):** O processo está na memória aguardando para receber tempo de CPU.
- **Encerrado (Terminated):** A execução foi finalizada.

3. Bloco de Controle de Processos (PCB)

Cada processo é representado no sistema operacional por um **PCB**, que funciona como um repositório de informações necessárias para gerenciar o processo. Ele armazena o estado atual, o contador de programa, os registradores da CPU, informações de escalonamento, limites de memória e o status de dispositivos de I/O alocados.

4. Escalonamento e Filas

Os processos migram entre diferentes filas gerenciadas pelo sistema operacional :

- **Fila de Jobs:** Todos os processos presentes no sistema.
- **Fila de Prontos:** Processos na memória principal aguardando execução.
- **Filas de Dispositivos:** Processos esperando por um dispositivo de I/O específico.

5. Tipos de Escalonadores

O sistema utiliza diferentes níveis de decisão para otimizar o fluxo de trabalho :

- **Escalonador de Longo Prazo (Jobs):** Determina quais processos entram no sistema (fila de prontos) e controla o grau de multiprogramação. É invocado com menos frequência.
- **Escalonador de Curto Prazo (CPU):** Seleciona qual processo pronto deve assumir a CPU imediatamente. Precisa ser extremamente rápido (milissegundos).
- **Escalonador de Médio Prazo:** Gerencia o **Swapping**, removendo processos da memória para o disco (swap out) e trazendo-os de volta (swap in) para aliviar a carga do sistema.

6. Troca de Contexto (Context Switch)

Quando o sistema alterna a CPU entre processos, ele realiza uma **Troca de Contexto**, salvando o estado do processo atual em seu PCB e carregando o estado do próximo processo. Esse procedimento é considerado um *overhead* (gasto extra), pois o sistema não realiza trabalho útil para o usuário durante a transição.

7. Comportamento do Processo

Os processos podem ser classificados em dois tipos :

- **Limitados por I/O (I/O-bound):** Passam mais tempo realizando operações de entrada/saída do que cálculos.
- **Limitados por CPU (CPU-bound):** Gastam a maior parte do tempo em processamento e cálculos intensos.

CAP 6

1. Conceitos Básicos e Ciclo de Execução

O objetivo central do escalonamento é obter a **utilização máxima da CPU** por meio da multiprogramação. A execução de um processo é descrita como um ciclo que alterna entre:

- **Surto de CPU (CPU Burst):** Período em que o processo realiza cálculos.
- **Surto de I/O (I/O Burst):** Período em que o processo aguarda a conclusão de operações de entrada e saída. A distribuição desses surtos geralmente apresenta um grande número de surtos curtos e poucos surtos longos.

2. Escalonador e Dispatcher

- **Escalonador de CPU:** Seleciona processos na memória que estão prontos para executar e aloca a CPU para um deles. As decisões ocorrem em quatro situações: mudança de execução para espera, execução para pronto, espera para pronto ou término do processo.
- **Preempção:** O escalonamento é **não-preemptivo** quando o processo mantém a CPU até terminar ou bloquear. É **preemptivo** quando o sistema pode interromper um processo em execução (ex: Unix).
- **Dispatcher (Executor):** Módulo que efetivamente dá o controle da CPU ao processo. Ele realiza a troca de contexto, muda para o modo usuário e salta para a posição correta no programa.

3. Critérios de Avaliação

O documento define métricas para comparar algoritmos :

- **Utilização da CPU:** Manter o processador ocupado o máximo possível.
- **Throughput:** Número de processos completados por unidade de tempo.
- **Tempo de Retorno (Turnaround):** Tempo total para executar um processo.
- **Tempo de Espera:** Tempo gasto na fila de prontos.
- **Tempo de Resposta:** Tempo entre a requisição e a primeira resposta gerada.

4. Algoritmos de Escalonamento

- **FCFS (First-Come, First-Served):** Simples (FIFO), mas pode causar o **Efeito Comboio**, onde processos curtos esperam por um longo, elevando o tempo médio de espera.
- **SJF (Shortest Job First):** Seleciona o processo com o menor surto de CPU seguinte. É considerado ótimo para minimizar o tempo médio de espera. Pode ser preemptivo (**SRTF - Shortest Remaining Time First**) ou não-preemptivo.
- **Prioridade:** Atribui um número inteiro a cada processo; a CPU vai para o de maior prioridade (menor valor). O problema de **Starvation** (inanição) ocorre quando processos de baixa prioridade nunca executam, sendo resolvido pelo **Envelhecimento (Aging)**.
- **Round Robin (RR):** Projetado para tempo compartilhado. Cada processo recebe um **quantum de tempo** (10-100 ms). Se não terminar no prazo,

volta para o fim da fila. Se o quantum for muito pequeno, o gasto com troca de contexto (*overhead*) torna-se excessivo.

5. Escalonamento em Filas

- **Filas Múltiplas:** A fila de prontos é dividida (ex: primeiro plano interativo e segundo plano em lote), cada uma com seu próprio algoritmo (RR ou FCFS).
- **Filas Múltiplas com Realimentação (MLFQ):** Permite que processos migrem entre filas. Se um processo usa muita CPU, é rebaixado; se espera muito, pode ser promovido (envelhecimento). É definido por parâmetros como número de filas e métodos de promoção/rebaixamento.

CAP 8

1. Programação Concorrente e Paralelismo

O documento diferencia a execução concorrente (processos com potencial para execução paralela) da programação concorrente, que envolve notações e técnicas para expressar esse paralelismo e resolver problemas de **sincronização e comunicação**. É apresentado o exemplo do algoritmo *Merge Sort*, onde a execução paralela de duas metades pode reduzir significativamente o custo computacional, desde que haja sincronização correta.

2. O Problema da Exclusão Mútua

A exclusão mútua é a abstração fundamental para evitar que dois processos acessem simultaneamente o mesmo recurso (seção crítica). Uma solução correta deve ser composta por quatro etapas :

- **Pré-protocolo:** Preparação para entrar na seção crítica.
- **Seção Crítica:** Acesso ao recurso compartilhado.
- **Pós-protocolo:** Liberação do recurso.
- **Comandos:** Restante do código do processo.

3. Tentativas de Solução via Software (*Busy Waiting*)

O material descreve a evolução lógica de tentativas de resolver a exclusão mútua usando espera ocupada (*busy waiting*), onde o processo testa continuamente uma variável :

- **1ª Tentativa (Alternância Estrita):** Usa uma variável turn. Garante exclusão mútua, mas obriga os processos a alternarem rigidamente, o que pode bloquear o sistema se um processo for muito mais lento ou parar.
- **2ª Tentativa (Flags de Estado):** Usa flags para indicar se um processo está na seção crítica. Falha por não garantir exclusão mútua (ambos podem entrar ao mesmo tempo).
- **3ª Tentativa (Sinalização de Intenção):** O processo sinaliza a intenção antes de testar o outro. Evita a sobreposição, mas pode causar **Deadlock** (ambos esperam o outro desistir).
- **4ª Tentativa (Recuo/Gentileza):** O processo retira sua intenção temporariamente se houver conflito. Embora evite o deadlock, pode levar ao **Lockout** (inanição), onde os processos ficam alternando intenções sem nunca progredir.

4. Algoritmo de Dekker

Apresentado como o primeiro algoritmo provadamente correto para exclusão mútua entre dois processos. Ele combina as flags de intenção com a variável turn (prioridade). Se ambos os processos desejam entrar, aquele que não tem a "sua vez" (turn) retira sua intenção e espera, garantindo que o outro prossiga sem gerar deadlock ou lockout.

5. Semáforos

Para evitar o desperdício de ciclos de CPU com espera ocupada, utilizam-se os **Semáforos**. Um semáforo é uma variável inteira não negativa que só pode ser acessada por duas operações atômicas :

- **Wait (s):** Se $s > 0$, decrementa s. Se $s = 0$, o processo é **suspenso** (bloqueado).
- **Signal (s):** Se houver processos suspensos, acorda um deles; caso contrário, incrementa s. Esta solução é considerada mais elegante e eficiente, pois o sistema operacional gerencia os processos bloqueados em filas.

6. O Problema do Produtor-Consumidor

Este é o problema clássico de coordenação onde um produtor gera dados para um buffer e um consumidor os retira. A sincronização correta via semáforos deve garantir que:

1. O consumidor não retire dados de um buffer vazio (**wait no semáforo de itens**).

2. O produtor não insira dados em um buffer cheio (**wait no semáforo de espaço**).
3. O acesso ao buffer seja mutuamente exclusivo (**uso de semáforo binário/mutex**). O documento alerta que a **ordem dos comandos wait** é crítica: inverter a ordem entre o controle de exclusão mútua e o controle de buffer pode levar ao deadlock.

PARALELO

1. Concorrência vs. Paralelismo

O documento define **concorrência** como a existência de processos com potencial para execução paralela, enquanto a **programação concorrente** envolve as notações e técnicas para expressar esse paralelismo e resolver problemas de comunicação e sincronização. É apresentado o exemplo do *Merge Sort*, onde a divisão do trabalho entre dois processos pode reduzir o custo computacional de $n^2/2$ para aproximadamente $n^2/8 + n$.

2. O Problema da Exclusão Mútua

A exclusão mútua é a abstração que garante que duas atividades não se sobreponham no acesso a um recurso compartilhado. O documento estrutura a solução deste problema em quatro etapas :

1. **Pré-protocolo:** Preparação para entrar na seção crítica.
2. **Seção Crítica:** Onde ocorre o acesso ao recurso.
3. **Pós-protocolo:** Liberação do recurso.
4. **Comandos:** Restante do código do processo.

3. Evolução das Soluções via Software (*Busy Waiting*)

O material detalha cinco tentativas de resolver a exclusão mútua usando variáveis compartilhadas e espera ocupada :

- **1ª Tentativa (Alternância Estrita):** Usa uma variável *turn*. Garante exclusão mútua, mas falha se um processo for muito mais lento ou abortar, bloqueando o outro indefinidamente.
- **2ª Tentativa (Flags de Estado):** Processos usam flags para indicar se estão na seção crítica. Falha porque ambos podem verificar as flags simultaneamente e entrar juntos.

- **3ª Tentativa (Sinalização de Intenção):** O processo sinaliza sua intenção (flag=1) antes de testar o outro. Evita a sobreposição, mas causa **Deadlock** se ambos sinalizarem ao mesmo tempo.
- **4ª Tentativa (Recuo ou Gentileza):** O processo retira sua intenção temporariamente se houver conflito. Evita o deadlock, mas pode levar ao **Lockout** (inanição), onde os processos alternam intenções sem nunca progredir.

4. Algoritmo de Dekker

Apresentado como a primeira solução provadamente correta para dois processos. Ele combina a sinalização de intenção com a variável turn. Se ambos querem entrar, o processo que não tem a prioridade (turn) retira sua intenção e espera sua vez, garantindo exclusão mútua, progresso e ausência de deadlocks.

5. Semáforos

Para superar a ineficiência da espera ocupada (*busy waiting*), que consome ciclos de CPU desnecessários, Dijkstra propôs os **Semáforos**. Um semáforo é um inteiro não negativo acessado por duas operações atômicas :

- **Wait (s):** Se o valor é 0, o processo é **suspenso/bloqueado** pelo sistema operacional até que um sinal ocorra.
- **Signal (s):** Acorda um processo suspenso ou incrementa o valor se não houver ninguém esperando.

6. Problema do Produtor-Consumidor

O documento encerra com a aplicação de semáforos no problema do buffer limitado. A sincronização exige:

- Um semáforo para contar itens disponíveis (evita que o consumidor aja em buffer vazio).
 - Um semáforo para controlar espaços vazios (evita que o produtor aja em buffer cheio).
 - Um semáforo binário (**mutex**) para garantir que apenas um processo manipule a estrutura do buffer por vez. **Nota de segurança:** O documento alerta que a inversão da ordem das operações wait (ex: colocar o mutex antes do contador de itens) pode levar ao **Deadlock**.
-

DEADLOCK

1. As Quatro Condições Necessárias

Para que um deadlock ocorra, quatro condições devem estar presentes simultaneamente:

- **Exclusão Mútua:** Pelo menos um recurso deve ser mantido em modo não compartilhável (apenas um processo por vez).
- **Posse e Espera (Hold and Wait):** Um processo que já detém pelo menos um recurso está aguardando para adquirir recursos adicionais que estão com outros processos.
- **Não Preempção:** Os recursos não podem ser retirados forçadamente de um processo; eles devem ser liberados voluntariamente após a conclusão da tarefa.
- **Espera Circular:** Existe uma cadeia de processos $\{P_0, P_1, \dots, P_n\}$ onde P_0 espera por um recurso de P_1 , P_1 por P_2 , e P_n espera por um recurso de P_0 .

2. Deadlock em Sincronização (Documento "Paralela")

No contexto de programação concorrente, o deadlock é frequentemente ilustrado por falhas de implementação:

- **Terceira Tentativa de Exclusão Mútua:** Ocorre quando dois processos sinalizam a intenção de entrar na seção crítica simultaneamente e ambos ficam esperando que o outro retire sua intenção, paralisando o sistema.
- **Problema do Produtor-Consumidor:** O deadlock acontece se a ordem das operações wait for é invertida. Se um consumidor executa o wait(mutex) antes do wait(full) quando o buffer está vazio, ele retém o acesso exclusivo (trava o mutex) e fica bloqueado esperando por um item que o produtor nunca conseguirá inserir, pois o mutex está ocupado.
- **Jantar dos Filósofos:** Um exemplo clássico onde cada filósofo pega o garfo à sua esquerda e espera pelo da direita; se todos fizerem isso ao mesmo tempo, ocorre uma espera circular.

3. Métodos de Tratamento

Existem quatro estratégias principais para lidar com deadlocks:

- **Prevenção:** Consiste em garantir que pelo menos uma das quatro condições necessárias nunca ocorra. Exemplo: exigir que o processo peça todos os recursos de uma vez ou use uma ordenação total de recursos.

- **Evitação (Avoidance):** O sistema decide dinamicamente se uma alocação é segura. O **Algoritmo do Banqueiro** é a técnica mais conhecida, usando informações sobre as necessidades máximas de cada processo para evitar estados de risco.
- **Detecção e Recuperação:** O sistema permite que o deadlock ocorra, identifica-o periodicamente (usando grafos de alocação de recursos) e tenta recuperar-se abortando processos ou retirando recursos (preempção).
- **Algoritmo do Avestruz:** Simplesmente ignorar o problema, assumindo que deadlocks são raros e que o custo de prevenção não compensa o benefício (estratégia comum em sistemas como Windows e Unix).

4. Grafo de Alocação de Recursos

É uma ferramenta visual para identificar deadlocks. Se o grafo contiver um ciclo e cada tipo de recurso tiver apenas uma instância, o deadlock é garantido. Se houver múltiplas instâncias por recurso, um ciclo indica apenas a *possibilidade* de deadlock.